

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

Ingegneria del Software 2

SOFTWARE TESTING IN APACHE STORM

Progettazione sistematica, adequacy, generazione automatica, reliability
e valutazione delle varianti C0-C4

Report finale

Studente: Valerio Torac

Progetto: Apache Storm

Baseline: Apache Storm 3.0.0 - tag v3.0.0

Repository: <https://github.com/BestOfTorac/storm/tree/isw2-project>

A.A.: 2025/2026

Settembre 2026

Indice

La struttura segue il percorso sperimentale: progettazione funzionale, crescita dell'adequacy, generazione automatica, valutazione C0-C4 e validità. Ogni voce dell'indice è un collegamento interno cliccabile.

1. Introduzione e selezione del SUT	3
2. Progettazione ed evoluzione della suite manuale	3
2.1 Category Partition	3
2.2 Adequacy e Control-Flow Testing	5
2.3 Mutation Testing e reliability	6
3. Test automatici	6
3.1 Randoop	7
3.2 EvoSuite	7
3.3 LLM: protocollo, prompt e freeze	7
3.4 Lettura comparativa delle suite automatiche	7
4. Valutazione delle varianti C0-C4	8
4.1 Contesto di refactoring e behavior preservation	8
4.2 Rigenerazione automatica e delta	8
4.3 Qualità e manutenibilità del codice di test	8
5. CI/CT e minacce alla validità	9
5.1 CI/CT e riproducibilità	9
5.2 Minacce alla validità	9
6. Conclusioni	9
Appendice A - Progettazione e risultati manuali	11
Appendice B - Test automatici e measurement contract	13
Appendice C - Confronto C0-C4	15
Appendice D - Tracciabilità	19

Le tabelle sono raccolte nelle appendici e richiamate dal corpo tramite collegamenti interni cliccabili. Non vengono inserite figure: la documentazione quantitativa e metodologica è concentrata in matrici e tabelle.

1. Introduzione e selezione del SUT

Una suite di test efficace non coincide con una suite che semplicemente passa. Deve rappresentare comportamenti rilevanti, attraversare decisioni significative e possedere oracle capaci di distinguere una deviazione del programma da un'esecuzione corretta. Questo studio osserva queste dimensioni su Apache Storm 3.0.0 e costruisce un percorso in cui ogni tecnica aggiunge un tipo di informazione diverso: prima il comportamento osservabile, poi la struttura, quindi la capacità di discriminare mutazioni e infine l'effetto del refactoring sulla conservazione del comportamento e sulla testabilità automatica.

Le due classi finali sono **UIHelpers** e **RedisFilterBolt**. **UIHelpers** è una classe ampia e trasversale, con 2389 TypeLOC, 108 metodi dichiarati e 77 smell nella baseline; combina formattazione, HTTP/JSON, Jetty/SSL/CORS, risorse, topologie, statistiche e operazioni REST. **RedisFilterBolt** è molto più compatta, ma concentra decisioni semanticamente nette: il data type Redis determina l'operazione da interrogare e l'esito decide se una tuple debba essere emessa, soltanto acknowledged oppure gestita come errore. Il contrasto rende utile il confronto perché mette le stesse tecniche di fronte a superfici API e livelli di complessità molto diversi.

La selezione è stata resa deterministica. Dalle 538 classi concrete della release viene applicato un filtro di non banalità fissato prima del ranking: TypeLOC almeno 32 e almeno due metodi dichiarati, costruttori esclusi. Restano 381 candidati, ordinati per NSMELLS decrescente e FQCN crescente. Con $V=22$ e $V \bmod 5=2$, il rank first+2 individua **UIHelpers**. Il candidato last-2, DefaultHttpCredentialsPlugin, viene invece archiviato dopo il freeze della sua baseline: i 10 **T_BB** raggiungevano già 30/30 linee e 16/18 branch, lasciando quasi nessun margine informativo alla successiva crescita strutturale. La scansione prosegue quindi verso l'interno dello stesso ranking fino a **RedisFilterBolt**, senza cambiare filtro o tie-break. La **Tabella 1** sintetizza candidati e motivazione della scelta finale.

La sostituzione del secondo target non è un tentativo di scegliere una classe "più facile", ma una decisione di suitability sperimentale: mantenere una baseline già saturata sulla line coverage avrebbe reso poco leggibile il contributo della fase control-flow, mentre rimuovere test validi per creare artificialmente un gap avrebbe compromesso la validità interna. L'evidenza del candidato archiviato rimane conservata proprio per rendere auditabile questa decisione.

Il principio metodologico che governa l'intero lavoro è la separazione tra costruzione ed evaluation. Una suite viene progettata o generata, verificata tecnicamente e congelata; soltanto dopo vengono osservate coverage, mutation e qualità strutturale. Lo stesso principio vale per le varianti **C1-C4**. Un punteggio basso, un no-op, un compile blocker o una regressione non autorizzano una rigenerazione opportunistica: sono risultati da interpretare.

2. Progettazione ed evoluzione della suite manuale

La suite manuale è costruita come una sequenza di diagnosi e risposta. **Category Partition** stabilisce che cosa merita un oracle senza conoscere i gap strutturali; dopo il freeze, **JaCoCo** rende visibili le decisioni interne ancora debolmente esercitate e apre **T_CF**; soltanto dopo questo secondo freeze **PIT** valuta se gli oracle distinguono piccole alterazioni del production code e, quando il survivor corrisponde a un comportamento osservabile, giustifica **T_MT**. La **Tabella 8** riassume la genealogia completa delle due classi.

2.1 Category Partition

Perché partire dalla Category Partition. La progettazione manuale inizia dal comportamento osservabile, non dal codice. **Category Partition** consente di trasformare le responsabilità pubbliche del SUT in categorie, classi di equivalenza, scelte, vincoli e boundary prima che coverage o mutazioni influenzino la selezione dei casi. Il vantaggio non è soltanto organizzativo: in questo modo la baseline **T_BB** diventa un riferimento funzionale indipendente. Quando, nelle fasi successive, un nuovo test verrà aggiunto perché un branch è scoperto o perché un mutante sopravvive, sarà possibile distinguere con precisione ciò che nasce dalla specifica da ciò che nasce dall'ispezione strutturale.

Categorie, frame e constraints. Per ogni responsabilità vengono individuate dimensioni di input o di stato capaci di modificare l'output esterno. Le scelte appartenenti alla stessa classe di equivalenza non vengono replicate meccanicamente; al contrario, i **constraints** eliminano combinazioni impossibili, prive di significato o semanticamente ridondanti. Il risultato non è quindi il prodotto cartesiano di tutte le scelte, ma un insieme di frame rappresentativi nei quali ogni variazione conservata corrisponde a una differenza che il test può realmente osservare. Questa regola è particolarmente importante su una classe ampia come **UIHelpers**, dove combinare indiscriminatamente configurazioni Jetty, callback, porte, collezioni e stati di topologia produrrebbe molti casi senza aumentare in modo proporzionale l'informazione della suite.

Gerarchia degli oracle. Un input interessante non è sufficiente: per entrare nella baseline deve essere accompagnato da un oracle difendibile. La precedenza viene data a contratti REST/configurazione, Javadoc, firme

e DTO pubblici. Quando la documentazione consente di affermare soltanto una proprietà parziale, il test verifica quella proprietà e non ricostruisce dettagli ulteriori dall'implementazione. Se invece l'output esatto è conoscibile soltanto leggendo **C0**, quel dettaglio non viene trasformato retroattivamente in requisito black-box: rimane fuori dalla baseline oppure viene affrontato più avanti, quando l'ispezione della struttura è metodologicamente ammessa. La provenienza degli oracle è resa esplicita in **Tabella 4**.

UIHelpers: scomposizione del dominio. La difficoltà principale è l'eterogeneità della classe. Trattarla come un unico blocco avrebbe mescolato domini con semantiche molto diverse; l'analisi la scompone quindi in **13 famiglie funzionali**, riportate in **Tabella 2**: formatting, URL/logviewer, JSON/HTTP, Jetty/SSL/filtri, cluster e risorse, supervisor/worker, topologie, statistiche, error representation, visualization, profiling, operazioni topology e Nimbus/logging. La presenza di una famiglia nell'analisi del dominio non implica automaticamente una test class dedicata: l'ingresso nella baseline richiede sempre che il frame selezionato abbia un comportamento pubblico distinguibile e un oracle sufficientemente solido.

Boundary funzionali in UIHelpers. I valori limite non vengono scelti perché "vicini" a un if visto nel codice, ma perché separano rappresentazioni o stati osservabili. Nelle trasformazioni temporali compaiono i passaggi 59/60 secondi e 999/1000 ms; per JSON/JSONP si distinguono callback assente, valida e non valida e si considera la soglia 128/129; per SSL e logviewer si separano porte assenti, negative, zero e positive; per le strutture di cluster e topologia si usano cardinalità 0/1/molti e stati di risorsa inutilizzata, parziale o piena. Questi boundary costituiscono il vocabolario del dominio; soltanto quelli per i quali esiste un oracle esterno netto entrano nella **T_BB strict**.

Esempi di derivazione dei casi. Il test **WH01** usa il valore speciale **:all-time** e verifica la rappresentazione "All time": la ragione del caso è funzionale e precede **JaCoCo**. **WH04** usa invece il boundary 60 secondi e verifica "1m 0s". Nel dominio JSON/HTTP un callback assente deve produrre una risposta JSON con Content-Type coerente; nelle operazioni topology l'oracle controlla proprietà pubbliche della risposta come operation, topologyId e status. Questi esempi mostrano che la Category Partition non sceglie semplicemente valori diversi: associa a ogni frame una proprietà osservabile che giustifica l'esistenza del test.

Audit degli oracle e baseline strict. Su **UIHelpers** questo audit è stato il passaggio più delicato. Alcune famiglie permettono di individuare facilmente condizioni limite, ma la documentazione non definisce sempre l'intera stringa o struttura restituita. In tali casi il boundary resta parte dell'analisi, ma l'aspettativa implementation-derived non viene usata per rendere artificiosamente più ricca la baseline. Il set finale contiene **35 T_BB** distribuiti su dieci test class, con la composizione riportata in **Tabella 5**. Il numero è quindi una conseguenza delle partizioni effettivamente rappresentabili con oracle esterni, non un target numerico deciso a priori.

Come si compongono i 35 test UIHelpers. La distribuzione riportata in **Tabella 5** aiuta a leggere il peso delle diverse responsabilità: formatting, URL/logviewer e JSON/HTTP hanno due test ciascuno; Jetty/SSL/filtri, cluster/risorse e supervisor/worker ne hanno sei ciascuno; la famiglia topology ne ha sette; statistiche due; operazioni topology e Nimbus/logging un caso dedicato ciascuna. La differenza non riflette una priorità arbitraria tra moduli, ma la quantità di stati esternamente distinguibili. Le famiglie di configurazione e di rappresentazione distribuite combinano più condizioni pubbliche - porte, protocollo, cardinalità, stato delle risorse, worker e topology - mentre alcune trasformazioni possono essere rappresentate da pochi frame parametrici con oracle molto netti.

Analisi del dominio e suite eseguibile non coincidono. Le 13 famiglie della **Tabella 2** costituiscono una mappa delle responsabilità e dei relativi domini; la **Tabella 5** descrive invece ciò che entra nella baseline strict. La differenza è intenzionale: riconoscere una responsabilità o un boundary non obbliga a creare un test se l'oracle esterno non è sufficientemente autonomo. Questa separazione evita due errori opposti: ignorare parti del dominio solo perché non sono immediatamente testabili e, all'estremo contrario, attribuire alla specifica un comportamento ricavato in realtà da C0. Le famiglie restano quindi parte dell'analisi, mentre **T_BB** conserva soltanto i frame difendibili come veri black-box.

Cardinalità e non ridondanza. Un singolo metodo JUnit può verificare più proprietà dello stesso frame quando esse appartengono allo stesso comportamento osservabile; viceversa, due combinazioni vengono tenute separate quando cambiano l'esito o il contratto verificato. Questa scelta evita due estremi: una suite minimale che perde classi di equivalenza importanti e una suite gonfiata da variazioni che ripetono lo stesso oracle. La cardinalità finale può quindi essere difesa test per test ricostruendo la catena **responsabilità → categoria → scelta/boundary → frame → oracle**, invece di giustificarla con la sola quantità.

RedisFilterBolt: due decisioni osservabili. Il dominio è più compatto, ma la partizione segue lo stesso principio. La prima decisione è il **data type Redis**, che determina quale collaborazione deve essere invocata; la seconda è l'esito semantico di presenza/assenza, che determina l'outcome del bolt. **Tabella 3** separa quindi ambiente Redis (JedisPool/JedisCluster), sei data type documentati, stato presente/assente, outcome forward+ack o filter+ack, boundary specifici HLL/GEO e delega degli output fields.

Semantica del dispatch. **STRING** usa **exists**, **HASH** **hexists**, **SET** **sismember**, **SORTED_SET** **zrank**, **HYPER_LOG_LOG** **pfcoun**t e **GEO** **geopos**. Il test non si ferma però a verificare che il mock sia stato chiamato:

l'oracle finale è l'effetto sul bolt. Un dato presente deve produrre **emit + ack**; un dato assente deve evitare l'emissione ma mantenere **ack**. I boundary 0/1 di HLL, rank/null per sorted set e assenza/presenza di coordinate GEO sono significativi perché fanno cambiare proprio questa decisione osservabile.

Isolamento con Mockito. JedisCommandsContainer e OutputCollector vengono mockati per rendere deterministiche le risposte Redis e le interazioni del bolt senza avviare un server reale. Il mock è quindi un confine di isolamento, non il comportamento che si vuole dimostrare. La suite finale di **11 T_BB** rappresenta entrambi gli ambienti, tutti i sei tipi, i due esiti semantici, i boundary più significativi e declareOutputFields. Anche qui il prodotto cartesiano completo viene escluso perché molte combinazioni replicherebbero lo stesso outcome osservabile senza introdurre una nuova scelta di categoria.

Lettura complessiva della baseline manuale. Le due classi arrivano così a baseline di dimensione diversa per ragioni legate al dominio, non per un criterio uniforme di cardinalità. **UIHelpers** richiede una scomposizione per famiglie e un audit severo degli oracle; **RedisFilterBolt** permette una partizione più compatta centrata sul dispatch e sugli effetti emit/ack. In entrambi i casi la baseline viene congelata prima di osservare i gap strutturali: solo da questo momento coverage e codice interno possono diventare nuove fonti informative.

2.2 Adequacy e Control-Flow Testing

Perché i black-box non bastano. Una suite derivata dal dominio può essere funzionalmente corretta e tuttavia non attraversare tutte le decisioni dell'implementazione. La Category Partition risponde alla domanda "quali comportamenti sono significativi?", ma non garantisce che ogni ramo usato per realizzarli venga esercitato. Per questo la fase successiva nasce soltanto dopo il freeze **T_BB**: la coverage viene usata come strumento diagnostico, non come criterio retroattivo per scegliere i casi black-box.

Metriche e cambio di prospettiva. Le due metriche strutturali primarie sono **Line Coverage** e **Branch Coverage**. La prima segnala quali linee della classe target vengono raggiunte; la seconda rende visibili le alternative dei predicati e può quindi scoprire gap anche all'interno di codice già parzialmente eseguito. Dopo la misura, la fonte informativa cambia deliberatamente: **T_CF** è white-box, può usare C0 e il report **JaCoCo**, ma estende senza modificare la baseline **T_BB**. Questo ruolo è distinto da EvoSuite: JaCoCo+T_CF è crescita manuale mirata, mentre EvoSuite sarà una generazione automatica search-based.

UIHelpers: diagnosi dei gap. I 35 **T_BB** coprono il **39,13% delle linee** e il **35,12% dei branch**. La distanza tra l'ampiezza del dominio rappresentato e la coverage ottenuta è coerente con una classe da migliaia di linee e responsabilità eterogenee: molti comportamenti pubblici vengono esercitati, ma rimangono alternative interne non attraversate. I dieci **T_CF** vengono quindi associati a gap specifici, tra cui scelta HTTP/HTTPS nei link di logviewer, validazione JSONP, fallback di configurazione, cluster non vuoto, serializzazione di eccezioni e trasformazioni annidate.

Esempi CF03 e CF05. In **CF03** la presenza di una porta HTTPS positiva forza il percorso secure del logviewer e consente di verificare il link https. **CF05** esercita invece il ramo di validazione della callback JSONP: una callback invalida o oltre la soglia, ad esempio 129 caratteri, deve ricadere sul plain JSON senza wrapper JSONP. La differenza rispetto a WH01/WH04 è metodologica prima ancora che tecnica: CF03 e CF05 esistono perché un ramo di C0 è stato osservato dopo il freeze, non perché tali input fossero necessari per definire la baseline funzionale.

Effetto dell'adequacy growth su UIHelpers. Con i dieci casi aggiuntivi la suite cumulativa sale a 45 test e raggiunge **43,85% Line** e **50,41% Branch**. Il delta di +4,72 punti sulla line coverage ma +15,29 sulla branch coverage è particolarmente informativo: i nuovi test non stanno semplicemente attraversando molte istruzioni nuove, ma soprattutto alternative decisionali all'interno di porzioni di codice che la baseline raggiungeva solo parzialmente. Il risultato è quindi coerente con l'obiettivo della fase.

RedisFilterBolt: chiusura dei rami residui. La baseline parte già da **88,89% Line** e **76,19% Branch**. Qui il report JaCoCo individua un insieme molto più circoscritto di decisioni residue. Quattro test **T_CF** sono sufficienti a completare i rami del dispatch e degli esiti fino a **45/45 linee** e **21/21 branch**, cioè 100% su entrambe le metriche. Il punto di arresto non viene imposto a priori: deriva dal fatto che, sulla classe target, non rimangono ulteriori gap line/branch da colmare.

Confronto tra i due target. La stessa metodologia conduce quindi a risultati diversi senza cambiare criterio. Su UIHelpers una saturazione completa sarebbe poco realistica e avrebbe rischiato di trasformare la coverage in obiettivo fine a sé stesso; su RedisFilterBolt il 100% emerge naturalmente con poche aggiunte mirate. **Tabella 6** rende quantitativa l'evoluzione, mentre **Tabella 8** collega per entrambe le classi il limite osservato, la nuova informazione ammessa e la decisione di test che ne consegue.

2.3 Mutation Testing e reliability

Perché introdurre il mutation testing. Dopo T_CF sappiamo che una parte maggiore del codice è stata raggiunta, ma non sappiamo ancora se gli oracle distinguerebbero una piccola deviazione del programma. PIT viene quindi applicato alla suite cumulativa T_BB+T_CF. Un mutante **killed** è stato discriminato da almeno un test; **survived** indica che la modifica rimane compatibile con gli oracle; **no coverage** significa che il mutante non è stato raggiunto. Per questa ragione **Mutation Score** e **Test Strength** vengono letti insieme: la prima considera l'intera popolazione, la seconda concentra l'attenzione sui mutanti effettivamente raggiunti.

Dal survivor al test: non un mapping meccanico. Un survivor non genera automaticamente un nuovo test. Prima viene analizzato il comportamento modificato e si verifica se la differenza sia osservabile tramite un contratto utile. Mutazioni equivalenti, differenze esclusivamente interne o oracle che dipenderebbero da dettagli privati non giustificano un caso. Quando invece il survivor espone una proprietà di regressione significativa, quel comportamento diventa la motivazione del nuovo T_MT. In questo modo PIT rafforza la suite senza trasformarla in una collezione di assert costruiti soltanto per “uccidere il mutante”.

UIHelpers: audit dei survivor. Sulla suite T_BB+T_CF PIT genera 338 mutanti: **115 killed**, **61 survived** e **162 no coverage**, con Mutation Score 34,02% e Test Strength 65,34%. I survivor vengono raggruppati per comportamento e l'audit individua otto proprietà per le quali è possibile formulare oracle osservabili. I nuovi casi riguardano, tra gli altri, boundary JSONP/serializzazione, configurazione HTTPS/Jetty, percentuali di risorse, SSL, CORS e boundary di porta. La suite finale passa così da 45 a **53 test**.

Effetto di T_MT su UIHelpers. Dopo l'evoluzione PIT registra **155 killed** e **25 survived**, con Mutation Score **45,86%** e Test Strength **86,11%**. La misura JaCoCo post-hoc sul set già congelato aggiunge un'ulteriore informazione: le linee restano 492/1122, cioè **43,85%**, mentre i branch salgono da 122/242 a 130/242, cioè **53,72%**. Gli otto T_MT aggiungono quindi zero nuove linee ma otto branch e soprattutto oltre venti punti di Test Strength. Questo è il caso più chiaro dell'intero percorso in cui structural coverage e forza degli oracle mostrano contributi diversi.

MT06 come esempio di oracle mutation-guided. Il test MT06 osserva la configurazione CORS e richiede che l'allowed origin resti “*”. L'oracle nasce dall'analisi di un survivor su C0, ma nella fase C0-C4 diventa un detector reale di regressione: C1, C2 e C4 espongono **null** dove la baseline espone “*”, mentre C3 conserva il comportamento. Il caso mostra perché la mutation analysis non viene trattata come un punteggio astratto: può rendere esplicito un contratto che le fasi precedenti non avevano ancora protetto.

RedisFilterBolt: survivor equivalente e punto di arresto. Con 15 test la suite T_BB+T_CF raggiunge tutti i 12 mutanti, ne uccide 11 e ne lascia uno survived. L'audit manuale classifica il residuo come equivalente rispetto al comportamento osservabile. Aggiungere un sedicesimo test soltanto per creare formalmente una T_MT separata non avrebbe aumentato la capacità discriminante e avrebbe violato il criterio usato su UIHelpers. Il set finale resta quindi di **15 test** con Mutation Score e Test Strength pari a **91,67%**. La fase mutation-guided è completata anche se non produce una nuova test class.

Una genealogia, non una crescita obbligatoria. Il confronto tra le classi chiarisce il criterio generale: “migliorare la suite dopo PIT” significa analizzare i survivor e aggiungere casi quando esiste un gap comportamentale testabile, non aumentare sempre la cardinalità. Su UIHelpers l'analisi giustifica otto nuovi oracle; su RedisFilterBolt l'analisi giustifica il punto di arresto. **Tabella 8** sintetizza questa genealogia e rende esplicita la diversa causa di ogni incremento.

Reliability sul set manuale finale. Solo dopo avere chiuso l'evoluzione manuale viene stimata la reliability rispetto a un profilo operativo uniforme. Ogni test riceve probabilità $p_i = 1/N$ e l'estimatore $R_{\hat{}}$ coincide con la quota di casi passati. **UIHelpers** passa 53/53 e **RedisFilterBolt** 15/15, quindi $R_{\hat{}}=1$ e $Q_{\hat{}}=0$ nei due profili finiti, come riportato in **Tabella 7**. Il significato è volutamente circoscritto: non sono stati osservati failure nei comportamenti rappresentati dalla suite finale; non si tratta di una garanzia di affidabilità perfetta per workload reali di Storm.

3. Test automatici

Randoop, EvoSuite e LLM vengono valutati come famiglie di generazione differenti, non come tre modi per arrivare alla stessa suite. Il sorgente prodotto viene congelato prima delle misure; compilazione, esecuzione, coverage, PIT e qualità del test code sono osservati dopo il freeze. Quando una tecnica produce una cardinalità diversa, l'output naturale viene conservato: potare **EvoSuite** dopo aver visto i test avrebbe trasformato una suite generata in una suite selezionata manualmente e avrebbe contaminato il confronto. La same-cardinality viene quindi usata dove non richiede editing post-hoc, per esempio T_BB/T_RND/T_LLM su **RedisFilterBolt**.

3.1 Randoop

Randoop rappresenta l'esplorazione random/semi-random dell'API. Su **C0** vengono congelati 35 test per **UIHelpers** e 11 per **RedisFilterBolt**. Il risultato più utile è il contrasto tra reachability e fault detection: **UIHelpers** RND passa 35/35 ma copre 10,25% line e 12,40% branch, con **Mutation Score** 9,47%; su Redis la method coverage arriva al 100%, ma la branch coverage è 4,76% e nessuno dei 12 mutanti viene ucciso.

Questi valori mostrano che costruire sequenze eseguibili e raggiungere metodi non garantisce oracle semanticamente forti. Le suite rimangono congelate con naming generator-style e verbosità reale del generatore; una pulizia manuale successiva avrebbe migliorato leggibilità ma avrebbe reso meno chiaro quanto del risultato appartenga effettivamente a **Randoop**.

3.2 EvoSuite

EvoSuite è il generatore automatico search-based guidato da obiettivi di coverage. La sperimentazione ha richiesto una separazione esplicita tra ambiente nativo e compatibilità: **EvoSuite** 1.2.0 non gestisce correttamente parte del bytecode Java 25 e alcune sezioni Multi-Release di dipendenze recenti. La soluzione usa un percorso Java 11 e un classpath compatibile senza modificare la semantica del SUT.

Su **UIHelpers C0 EvoSuite** ottiene la migliore coverage automatica, 23,02% line e 22,41% branch, ma passa 40/43; **PIT** è quindi **N/A** perché non esiste una baseline fully passing. Su Redis **C0** la suite 9/9 è valida, con 35,56% line, 19,05% branch, **Mutation Score** 16,67% e **Test Strength** 66,67%. Generation success, test compile, test execution e **PIT** validity restano stati distinti.

3.3 LLM: protocollo, prompt e freeze

Microsoft 365 Copilot viene trattato come un generatore non deterministico che richiede controllo esplicito del contesto. Provider, modalità di accesso, prompt, file forniti, output, repeatability e SHA-256 vengono conservati. Il punto non è dimostrare che un singolo prompt sia "migliore", ma rendere ricostruibile quale informazione il modello possedeva quando ha prodotto una suite.

Su **UIHelpers** il percorso sperimenta più forme di interrogazione, riassunte nella **Tabella 10**. P1 combina analisi delle responsabilità e generazione iniziale; P2 è una critical review/refinement che preserva P1 come artefatto separato e non riceve coverage o mutation feedback; P3 rappresenta un'adequacy evolution controllata successiva alla baseline, anch'essa salvata senza sovrascrivere gli stadi precedenti. Questa separazione rende leggibile il passaggio da generation a refinement e poi, solo quando previsto dal protocollo, a feedback post-baseline.

Su **RedisFilterBolt** viene invece usata una strategia più compatta: un prompt strutturato chiede analisi funzionale ed esattamente 11 test con production-only context. Il primo tentativo viene invalidato perché Copilot consulta autonomamente una fonte Web esterna; l'output non viene corretto o ripulito, ma conservato come evidence non valida. Una nuova conversazione con divieto esplicito di fonti esterne produce la suite accettata, che passa 11/11 e cinque esecuzioni consecutive senza repair.

Il risultato **C0** conferma che il comportamento del generatore dipende fortemente dal target. Su **UIHelpers** LLM passa 35/35 e ha **Test Strength** 98% pur con **Line Coverage** 7,13%: gli oracle sono molto forti sulla porzione raggiunta, ma la reachability è limitata. Su Redis il profilo è più equilibrato: 84,44% line, 80,95% branch e 75% sia di **Mutation Score** sia di **Test Strength**.

Il protocollo impedisce di usare i risultati di una tecnica come suggerimento silenzioso per riscriverne un'altra. La **Tabella 10** rende visibile quando il feedback è vietato e quando un'evoluzione post-baseline è esplicitamente separata dall'artefatto originario.

3.4 Lettura comparativa delle suite automatiche

La **Tabella 9** confronta le metriche **C0**, mentre la **Tabella 12** sintetizza la rilevanza funzionale osservata. Il dato più importante è che structural coverage e fault detection possono divergere: **EvoSuite UIHelpers** raggiunge più struttura ma non fornisce una baseline fully passing per **PIT**; LLM **UIHelpers** raggiunge meno codice ma discrimina quasi tutti i mutanti raggiunti; **Randoop** è stabile ma più debole su entrambe le dimensioni.

Il caso Redis è ancora più netto. **Randoop** raggiunge tutti i metodi ma ha **Mutation Score** 0; **EvoSuite** raggiunge pochi mutanti ma mostra **Test Strength** 66,67% su quelli reached; LLM mappa meglio il dispatch e gli outcome emit/ack, raggiunge tutti i 12 mutanti **C0** e ne uccide 9. La "rilevanza" non viene quindi inferita dal numero di test, ma dalla relazione tra comportamento esercitato, oracle e risultato dinamico.

Chiarezza e manutenibilità costituiscono una terza dimensione. Naming, Assertions/Test, TestLOC/LogicLOC/SupportLOC e Sonar vengono misurati con regole uniformi. La **Tabella 11** definisce il measurement contract e chiarisce perché nessuna singola metrica viene trasformata in un punteggio composito.

4. Valutazione delle varianti C0-C4

La seconda parte dell'esperimento usa i test come contesto del refactoring. **C0** è la classe originale; **C1** viene refactorizzata senza test; **C2** riceve **T_BB**; **C3** **T_BB+T_CF**; **C4** il set manuale mutation-guided. Le varianti sono generate indipendentemente dalla stessa **C0** e non in catena, così che una regressione di una condizione non diventi automaticamente input della successiva.

RND, ES e LLM non vengono forniti al refactoring. Sono rigenerati solo dopo il freeze di ciascuna variante, con un obiettivo distinto: osservare come il cambiamento strutturale del production code influenzi la testabilità automatica. Il contesto crescente non viene interpretato come un ordine atteso di qualità: più test disponibili non garantiscono una variante migliore.

4.1 Contesto di refactoring e behavior preservation

La **Tabella 13** mostra quali suite entrano nel contesto **C0-C4**. Dopo la generazione, le suite **C0** vengono rieseguite senza editing sulle varianti: questa è la verifica di behavior preservation. La domanda è semplice e indipendente dalle nuove suite generate: un comportamento già protetto continua a essere soddisfatto?

Su **UIHelpers**, la **Tabella 14** mostra che **T_BB** 35/35 e **T_CF** 45/45 passano su tutte le varianti. La suite finale di 53 test individua invece una regressione selettiva: **C1**, **C2** e **C4** passano 52/53, mentre **C3** conserva 53/53. Il fallimento è **MT06**: **C0** espone allowed origin "*", mentre le varianti problematiche espongono null. La quantità di contesto non produce quindi una progressione monotona.

Su **RedisFilterBolt**, la **Tabella 15** mostra preservation completa per BB, CF/MT, LLM, RND ed ES su **C1-C4**. Il contrasto con **UIHelpers** è utile: una classe compatta e mockable conserva tutti gli oracle osservati, mentre la classe UI espone una regressione che diventa visibile solo grazie all'evoluzione mutation-guided.

4.2 Rigenerazione automatica e delta

Dopo il freeze di ogni variante vengono generate nuove suite RND, ES e LLM. La campagna contiene 2 classi x 5 versioni x 3 tecniche = 30 unità. I delta vengono calcolati rispetto alla **C0** della stessa tecnica, non tra tecniche differenti, perché cardinalità e scaffolding non sono omogenei. **PIT** è calcolato solo quando la suite è fully passing; **N/A** non viene sostituito con zero.

La **Tabella 16** e la **Tabella 17** riportano i delta completi. Su **UIHelpers Randoop** è stabile ma debole; **EvoSuite** raggiunge fino a 25,27% line e 25,21% branch su **C4**, ma nessuna unità UI-ES possiede un **Mutation Score** valido; LLM mantiene naming descrittivo ma sulle varianti diventa fragile, con compile blocker in **C1/C4** e suite non fully passing in **C2/C3**.

Su **RedisFilterBolt Randoop** aumenta la line coverage ma rimane a **Mutation Score** 0. **EvoSuite** migliora la line coverage in varie condizioni, ma nelle varianti **PIT** comparabili la mutation effectiveness crolla a 0. LLM mostra il profilo più forte: **C4** raggiunge 86,96% branch e 92% di **Mutation Score/Test Strength**, pur perdendo line coverage rispetto a **C0**.

Le 10 unità **PIT N/A** sono elencate nella **Tabella 18**. Mantenerle visibili evita due errori opposti: interpretarle come zero falserebbe la fault detection; eliminarle nasconderebbe la fragilità operativa. La matrice completa rimane quindi informativa anche quando una misura non è metodologicamente disponibile.

4.3 Qualità e manutenibilità del codice di test

Oltre all'efficacia dinamica delle suite, è stata valutata anche la qualità del codice di test generato. Le due dimensioni vengono mantenute separate: Line Coverage, Branch Coverage, Mutation Score e Test Strength misurano quanto efficacemente una suite esercita il SUT e rileva deviazioni, mentre naming, numero di assertion, quantità di codice e risultati Sonar descrivono quanto il test code sia leggibile e manutenibile.

In particolare, DescriptiveNamePct misura la quota di test con nomi semanticamente descrittivi; Assertions/Test indica la densità media di verifiche esplicite; TestLOC, LogicLOC e SupportLOC distinguono il codice effettivamente dedicato alla logica dei test dallo scaffolding necessario alla loro esecuzione. Sonar completa l'analisi mediante code smell, technical debt e categorie Clean Code. La **Tabella 11** riassume il significato di queste metriche e le regole con cui vengono confrontate.

Gli aggregati Sonar delle 30 unità sperimentali sono riportati nella **Tabella 19**. Randoop presenta complessivamente 1184 code smell e 10888 minuti di technical debt, EvoSuite 499 smell e 6103 minuti, mentre LLM 96 smell e 1902 minuti. L'analisi Sonar classifica inoltre i rilievi secondo gli attributi Clean Code, con 208 occorrenze CONSISTENT, 1316 INTENTIONAL, 255 ADAPTABLE e 0 RESPONSIBLE. Questi valori non vengono

però interpretati come una classifica assoluta delle tecniche, poiché i generatori producono quantità molto diverse di codice e scaffolding.

Il caso RedisFilterBolt C4 mostra bene il trade-off. La suite LLM raggiunge l'86,96% di Branch Coverage e il 92% sia di Mutation Score sia di Test Strength, risultando quindi molto efficace sul piano dinamico. Rispetto alla baseline C0, tuttavia, aumentano anche code smell e technical debt del test code. Il risultato mostra che fault detection e manutenibilità non necessariamente migliorano insieme: una suite può diventare più efficace nel rilevare deviazioni e, allo stesso tempo, più costosa da mantenere.

5. CI/CT e minacce alla validità

5.1 CI/CT e riproducibilità

Il modulo isw2/testing isola le suite del progetto dai test nativi di Storm e viene verificato tramite Maven e GitHub Actions. La CI controlla build, suite dedicate e integrità degli artefatti strutturati; hash e source map permettono di verificare che lo stesso sorgente congelato alimenti coverage, mutation e analisi del test code.

Le difficoltà tecniche non vengono trattate come rumore operativo, ma come potenziali minacce alla validità dell'infrastruttura. La **Tabella 22** raccoglie i casi principali: incompatibilità **EvoSuite**/Java, parser **PIT**, contaminazione LLM Redis, **N/A**, regressione CORS e misura post-hoc della coverage finale **UIHelpers**. In ogni caso la soluzione corregge l'ambiente o la lettura senza riscrivere retroattivamente una suite congelata.

Questa disciplina distingue un failure del framework da un failure del test e impedisce che un risultato dipenda solo dallo stato locale della workstation. La riproducibilità viene quindi trattata come relazione verificabile tra sorgente, ambiente, esito e misura, non come semplice conservazione dei file.

5.2 Minacce alla validità

Selection validity. Le due classi non sono un campione casuale e **RedisFilterBolt** sostituisce un candidato risultato poco informativo dopo il freeze **T_BB**. Ranking, filtro e tie-break restano invariati e l'evidence del candidato è conservata, ma la scelta di suitability limita comunque la generalizzabilità.

Construct validity. Line/Branch misurano reachability, non correttezza; **Mutation Score** dipende dalla popolazione di mutanti; **Test Strength** esclude i no-coverage; naming e assertion-like sono proxy parziali della leggibilità. Sonar dipende da LOC e scaffolding. La mitigazione è mantenere le dimensioni separate e dichiarare il measurement contract.

Internal validity. Il rischio principale è il feedback post-hoc. Freeze, SHA-256 e separazione temporale tra **T_BB**, **T_CF** e **T_MT** riducono l'ottimizzazione retroattiva. La misura **JaCoCo** finale dei 53 test **UIHelpers** è esplicitamente post-hoc e non ha guidato la progettazione dei **T_MT**.

Tool e LLM validity. **EvoSuite** richiede un ambiente Java 11 di compatibilità; **Microsoft 365 Copilot** è non deterministico e può cambiare. Prompt, contesto, provider, anomalie e hash migliorano la tracciabilità ma non trasformano una singola generazione in una stima universale della tecnica.

Conclusion ed external validity. **PIT** è misurabile per 20/30 unità e solo 15/24 delta di variante sono comparabili. Le conclusioni restano descrittive per due classi e per le generazioni osservate. Repliche su più target, seed e generazioni sarebbero necessarie per inferenze esterne più forti.

6. Conclusioni

Il risultato principale è che la qualità di una suite emerge dall'interazione tra **rilevanza funzionale, esplorazione strutturale e forza degli oracle**. La **Category Partition** definisce quali comportamenti meritano di essere protetti senza lasciarsi guidare dalla struttura dell'implementazione; **T_CF** usa **JaCoCo** per individuare decisioni ancora debolmente esercitate; **PIT** verifica infine se gli oracle siano effettivamente capaci di distinguere piccole deviazioni del programma. Le tre tecniche non sono quindi alternative, ma rispondono a domande differenti e acquistano valore proprio quando vengono applicate in sequenza.

L'evoluzione di **UIHelpers** rende evidente questa complementarità. La **Branch Coverage** cresce dal 35,12% della baseline black-box al 50,41% dopo il Control-Flow Testing e raggiunge il 53,72% con il set manuale finale. Parallelamente, la **Test Strength** passa dal 65,34% all'86,11%. Gli otto test mutation-guided non aggiungono nuove linee coperte, ma attraversano otto branch ulteriori e rafforzano in modo significativo la capacità degli oracle

di rilevare alterazioni. **RedisFilterBolt** raggiunge invece naturalmente il 100% di Line e Branch Coverage già con quattro T_CF aggiuntivi; PIT mostra poi che l'unico survivor residuo è equivalente rispetto al comportamento osservabile e non giustifica la costruzione artificiale di un ulteriore test. Le due classi raggiungono quindi punti di arresto diversi per ragioni sperimentali osservabili, non per una cardinalità decisa a priori.

Anche il confronto tra le tecniche automatiche mostra profili complementari. **Randoop** costituisce una baseline utile per l'esplorazione dell'API, ma può raggiungere molti metodi senza produrre oracle sufficientemente forti. **EvoSuite** spinge maggiormente la reachability strutturale, pagando però costi di scaffolding, compatibilità e stabilità della suite. L'approccio **LLM** può invece produrre test semanticamente mirati e oracle leggibili quando il comportamento è inferibile dal contesto, come mostra il dispatch di RedisFilterBolt, ma non garantisce né un'esplorazione sistematica del codice né una stabilità uniforme sulle varianti. Non emerge quindi una tecnica universalmente migliore: il risultato dipende dal tipo di SUT e dalla proprietà che si intende privilegiare.

La sperimentazione **C0-C4** mostra inoltre che fornire quantità crescenti di informazione di testing a un processo di refactoring non produce automaticamente una progressione monotona della qualità. **UIHelpers C3** è l'unica variante che preserva 53/53 test della suite manuale finale, mentre **C1, C2 e C4** perdono il comportamento protetto da **MT06**. RedisFilterBolt preserva invece tutti i comportamenti osservati, ma le suite RND, ES e LLM reagiscono in modo differente alle variazioni strutturali. **Testability, maintainability e fault detection possono quindi migliorare o peggiorare indipendentemente**, e nessuna di queste dimensioni può essere sostituita da un singolo indicatore.

Un secondo risultato importante riguarda il modo in cui devono essere interpretate le metriche. **Coverage elevata non implica automaticamente oracle forti, Mutation Score elevato non descrive la manutenibilità del test code e un numero ridotto di smell non garantisce una migliore fault detection.** Per questo nel progetto Line Coverage, Branch Coverage, Mutation Score, Test Strength, naming, assertion density, dimensione del test code e indicatori Sonar sono mantenuti distinti. La loro combinazione produce una lettura più completa della suite senza ridurla artificialmente a un punteggio unico.

Dal punto di vista operativo, il processo che emerge come più robusto è quello in cui **ogni test possiede una genealogia riconoscibile**: nasce da una partizione del dominio, da un gap strutturale oppure da una mutazione che espone una proprietà osservabile non ancora protetta. Freeze degli artefatti, hash, stati **N/A** espliciti, distinzione tra framework failure e test failure e regole di comparabilità impediscono che i risultati vengano corretti retroattivamente per adattarsi alle aspettative. Questa disciplina è particolarmente importante quando entrano in gioco strumenti automatici e modelli generativi, perché permette di distinguere il risultato prodotto dallo strumento dall'effetto di eventuali interventi successivi.

Il valore del progetto non consiste quindi soltanto nell'aver costruito e confrontato diverse suite di test, ma nell'aver trasformato il testing in **un processo sperimentale osservabile e riproducibile**. Ogni tecnica aggiunge un tipo specifico di evidenza: la Category Partition descrive il comportamento che interessa preservare; il Control Flow mostra ciò che la specifica non riesce a raggiungere; il Mutation Testing misura quanto gli oracle siano discriminanti; i generatori automatici mostrano come strategie differenti esplorino lo stesso SUT; la fase C0-C4 verifica infine se tali evidenze siano sufficienti a proteggere trasformazioni automatiche del production code.

In un sistema reale come Apache Storm questo aspetto è particolarmente rilevante: **una suite non è utile soltanto quando segnala che il codice è corretto, ma quando permette di capire quali proprietà sono state verificate, quali rimangono deboli e quanto sia affidabile una trasformazione successiva**. Il contributo principale della sperimentazione è quindi una metodologia per combinare tecniche manuali, metriche strutturali, mutation analysis e generazione automatica senza confonderne il ruolo, mantenendo ogni risultato — positivo o negativo — tracciabile fino alla decisione che lo ha prodotto.

Appendice A - Progettazione e risultati manuali

Le tabelle di questa appendice rendono verificabile il percorso di progettazione: selezione dei target, **Category Partition**, composizione delle suite manuali, provenienza degli oracle, crescita dell'adequacy e reliability. I richiami nel corpo del report sono collegamenti interni cliccabili.

Tabella 1 - Selezione delle classi e profilo dei target

Classe	Posizione	TypeLOC	Metodi	NSMELLS	Stato
UIHelpers	rank 3 / first+2	2389	108	77	Target finale
DefaultHttpCredentialsPlugin	rank 379 / last-2	62	3	0	Archiviato: T_BB line 100%
RedisFilterBolt	rank 362 / last-19	107	2	0	Target finale

Tabella 2 - UIHelpers: famiglie della Category Partition, partizioni e boundary

Fam.	Area	Partizioni / scelte principali	Boundary / valori rappresentativi
F1	Formatting	uptime sec/ms; range executor; window hint; sanitizzazione	59/60 s; 999/1000 ms; :all-time; input valido/non valido
F2	URL e logviewer	numero/tipo argomenti; HTTP/HTTPS; porte; host; filename	HTTPS assente/-1/0/8443; filename con spazi; HTTP vs HTTPS
F3	JSON e HTTP	callback; sintassi; lunghezza; header; serialize; status	callback <128, =128, 129; null; valida/invalida
F4	Jetty, SSL, filtri	porte; host; binding; trust store; client auth; CORS; filtri	porta null/0/positiva; SSL -1/0/8443; 0/1/più filtri
F5	Cluster e risorse	supervisor/topology; risorse; generic resources; owner	conteggi 0/1/molti; risorse 0/parziale/pieno
F6	Supervisor e worker	slot; blacklist; risorse; worker list; link log	used>total; worker 0/1/molti; HTTP/HTTPS
F7	Topology summaries	identity/state; resources; worker; spout/bolt; debug/history	0/1/molti; :all-time/numeric; debug off/on
F8	Statistiche componenti	window; mappe; executor; Bolt/Spout; system components	map empty/nonempty; executor absent/present
F9	Error representation	messaggio; status; lista errori; log endpoint	empty/null/nonempty; client/server; last error absent/present
F10	Topology visualization	topology; executor; system comp.; stream; box	0,1,<4,=4,>=5 box; stream ordinario/system
F11	Profiling	azioni pendenti; timeout; host:port; timestamp	0/1/molte azioni; timeout 0/+; porta non numerica
F12	Topology operations	activate/deactivate/rebalance/kill/debug; wait; sampling	sampling 0/+ /100; wait 0/+
F13	Nimbus e logging	logger; timeout; leader; seeds; protocollo	0/1/molti; leader/nonleader; HTTP/HTTPS

Tabella 3 - RedisFilterBolt: categorie, scelte, vincoli e oracle

Categoria	Scelte / range	Vincoli / motivazione	Oracle osservabile
C1 - Ambiente	JedisPool; JedisCluster	Entrambi i costruttori pubblici sono rappresentati	stesso contratto di filtro
C2 - Data type	STRING; HASH; SET; SORTED_SET; HYPER_LOG_LOG; GEO	chiave aggiuntiva valida dove prevista	operazione Redis coerente con il tipo
C3 - Esistenza	presente; assente	semantica dipendente dal data type	esiti distinguibili
C4 - Outcome	forward+ack; filter+ack	presenza -> emit+ack; assenza -> no emit+ack	interazioni OutputCollector
Boundary	HLL 0/1; SORTED_SET rank/null; GEO 0/>=1 coordinate	confini che cambiano la decisione	filter+ack vs emit+ack
C5 - Output fields	delega al mapper	un frame sufficiente	declareOutputFields delegato

Tabella 4 - Test rappresentativi, oracle e fonte dell'oracle

Test ID	Categoria	Input rappresentativo	Oracle	Fonte oracle
WH01	Window special	:all-time	"All time"	Documentazione REST / contratto pubblico; T_BB
WH04	Boundary window	60	"1m 0s"	Documentazione REST; T_BB
HDR01	JSON senza callback	callback=null	Content-Type application/json	Contratto REST; T_BB
DAEMON01	Nimbus logviewer	host + porta HTTP 8000	URL HTTP con nimbus.log	REST/configurazione pubblica; T_BB
OPRSP01	Topology operation	topology-activate + activate	operation/id/status=success	REST API; T_BB
RFB-BB-01	STRING presente	JedisPool; exists=true	emit + ack; no fail	Javadoc/API RedisFilterBolt; T_BB
RFB-BB-07	HLL assente	pfcount=0	no emit + ack	Javadoc + Category Partition; T_BB
RFB-BB-08	HLL presente	pfcount=1	emit + ack	Javadoc + Category Partition; T_BB
RFB-BB-11	Output fields	mapper + declarer	delega declareOutputFields	API pubblica; T_BB
CF03	Logviewer secure	HTTPS port=8443	porta 8443; link https	C0 + gap JaCoCo dopo freeze T_BB
CF05	JSONP invalido/oltre soglia	callback invalida; 129 caratteri	plain JSON; nessun wrapper JSONP	C0 + gap JaCoCo dopo freeze T_BB
MT06	CORS	corsFilterHandle()	allowed origin ""	C0 + survivor PIT; mutation-guided

La provenienza dell'oracle cambia per fase: **T_BB** usa contratti esterni; **T_CF** può usare **C0** e i gap **JaCoCo** dopo il freeze; **T_MT** può usare survivor **PIT**.

Tabella 5 - UIHelpers: composizione della baseline T_BB congelata

Famiglia / test class	Responsabilità osservata	N test	Criterio di selezione / boundary
F1 - Formatting	UIHelpersFormattingBBTest	2	:all-time e finestra 60 s con oracle pubblico
F2 - URL e logviewer	UIHelpersUrlLogviewerBBTest	2	protocollo/link osservabile e parametri documentati
F3 - JSON e HTTP	UIHelpersJsonHttpBBTest	2	JSON/JSONP e Content-Type osservabile
F4 - Jetty, SSL e filtri	UIHelpersJettySslFilterBBTest	6	porta/host/header, SSL e filtri con proprietà pubbliche
F5 - Cluster e risorse	UIHelpersClusterResourceBBTest	6	cluster vuoto, owner/resource summaries, cardinalità 0/1/molti
F6 - Supervisor e worker	UIHelpersSupervisorWorkerBBTest	6	summary, worker, slot e collezioni osservabili
F7 - Topology	UIHelpersTopologyBBTest	7	summary, worker locations, history e valori documentati
F8 - Statistiche	UIHelpersStatsComponentsBBTest	2	finestra :all-time e finestra numerica
F12 - Operazioni topology	UIHelpersTopologyOperationBBTest	1	activate: operation, topologyId, status
F13 - Nimbus e logging	UIHelpersNimbusLoggingBBTest	1	leader Nimbus: host, port e status
Totale T_BB UIHelpers	10 test class congelate	35	nessun feedback JaCoCo/PIT usato nella selezione

Tabella 6 - Evoluzione delle suite manuali e adequacy

Classe	Suite	Test	Line	Branch	Mutation Score	Test Strength
UIHelpers	T_BB	35	39,13%	35,12%	-	-
UIHelpers	T_BB+T_CF	45	43,85%	50,41%	34,02%	65,34%
UIHelpers	Finale con T_MT	53	43,85%	53,72%	45,86%	86,11%

Classe	Suite	Test	Line	Branch	Mutation Score	Test Strength
RedisFilterBolt	T_BB	11	88,89%	76,19%	N/A	N/A
RedisFilterBolt	T_BB+T_CF	15	100%	100%	91,67%	91,67%
RedisFilterBolt	Finale mutation-guided	15	100%	100%	91,67%	91,67%

Per **UIHelpers**, Line e Branch della suite finale da 53 test sono una misura **JaCoCo** post-hoc sul set già congelato: 492/1122 linee e 130/242 branch. La misura non è stata utilizzata per progettare **T_MT**.

Tabella 7 - Reliability sul profilo operativo uniforme

Classe	N	p per test	Pass	Fail	R_hat	Q_hat
UIHelpers	53	1/53 = 0,018868	53	0	1,000000	0,000000
RedisFilterBolt	15	1/15 = 0,066667	15	0	1,000000	0,000000

Tabella 8 - Genealogia dell'evoluzione manuale: limite, informazione e decisione

Classe / passaggio	Limite osservato	Nuova informazione ammessa	Decisione	Effetto
UIHelpers: specifica -> T_BB	API ampia e responsabilità eterogenee	contratti REST/configurazione e DTO pubblici	Category Partition per famiglie, boundary e oracle	35 test; Line 39,13%; Branch 35,12%
UIHelpers: T_BB -> T_CF	alternative interne ancora poco esercitate	C0 + JaCoCo dopo freeze T_BB	10 test su branch/gap mirati	45 test; Line 43,85%; Branch 50,41%
UIHelpers: T_CF -> T_MT	coverage non misura la forza degli oracle	survivor PIT dopo freeze T_CF	8 test mutation-guided su comportamenti osservabili	53 test; Branch 53,72%; MS 45,86%; TS 86,11%
Redis: specifica -> T_BB	dispatch dipendente da data type e outcome	Javadoc/API + Category Partition	11 frame su ambienti, 6 data type, presenza/assenza e boundary	11 test; Line 88,89%; Branch 76,19%
Redis: T_BB -> T_CF	rami residui pur con coverage già alta	C0 + JaCoCo dopo freeze T_BB	4 test mirati ai gap residui	15 test; Line 100%; Branch 100%
Redis: T_CF -> mutation	serve verificare la discrimination con coverage completa	PIT su T_BB+T_CF	audit del survivor residuo; nessun T_MT artificiale	11/12 killed; MS=TS=91,67%

Appendice B - Test automatici e measurement contract

Queste tabelle separano performance dinamica, protocollo di generazione, comparabilità e significatività funzionale delle suite automatiche.

Tabella 9 - Suite automatiche sulla baseline C0

Classe	Tecnica	Pass	Line	Branch	MS	TS	Smell	Debt min
UIHelpers	RND	35/35	10,25%	12,40%	9,47%	45,07%	183	1432
UIHelpers	ES	40/43	23,02%	22,41%	N/A	N/A	84	1023
UIHelpers	LLM	35/35	7,13%	15,29%	14,50%	98,00%	0	0
RedisFilterBolt	RND	11/11	48,89%	4,76%	0,00%	0,00%	70	806
RedisFilterBolt	ES	9/9	35,56%	19,05%	16,67%	66,67%	28	247
RedisFilterBolt	LLM	11/11	84,44%	80,95%	75,00%	75,00%	3	301

Tabella 10 - Protocollo LLM: fasi, contesto e controllo del feedback

Classe / fase	Contesto ammesso	Scopo	Feedback di adequacy	Esito metodologico
UIHelpers P1	production context + harness/POM necessario	analisi delle responsabilità e generazione iniziale	No	artefatto iniziale congelato e fingerprintato
UIHelpers P2	P1 + richiesta di critical review/refinement	rafforzare chiarezza e coerenza senza tuning su metriche	No coverage/mutation feedback	P1 preservato; P2 salvato separatamente

Classe / fase	Contesto ammesso	Scopo	Feedback di adequacy	Esito metodologico
UIHelpers P3	baseline precedente + informazione ammessa dal protocollo di adequacy evolution	verificare un'evoluzione controllata senza sovrascrivere la baseline	Sì, solo nella fase post-baseline prevista	P3 resta artefatto derivato; P1/P2 restano auditabili
RedisFilterBolt P1 - tentativo 1	production context previsto	generazione strutturata di 11 test	No	invalidato: uso autonomo di fonte Web esterna
RedisFilterBolt P1 - tentativo 2	nuova conversazione, tre file forniti, divieto esplicito di fonti esterne	analisi + esattamente 11 test	No	11/11 PASS; 5 run consecutive; freeze senza repair

Tabella 11 - Measurement contract delle suite automatiche e C0-C4

Dimensione	Indicatore	Interpretazione	Regola di comparabilità
Adequacy strutturale primaria	Line Coverage	quota di linee raggiunte	stesso target e denominatore
Adequacy strutturale primaria	Branch Coverage	alternative decisionali attraversate	stesso target e denominatore
Fault detection	Mutation Score	mutanti killed sull'intera popolazione	solo suite fully passing; N/A != 0
Forza degli oracle	Test Strength	killed sui mutanti raggiunti	solo popolazioni PIT valide
Validità operativa	compile + Pass/Exec	prerequisito per le misure dinamiche	framework failure distinto da test failure
Chiarezza	DescriptiveNamePct	naming semantico vs generator-style	stessa regola su tutte le tecniche
Densità degli oracle	Assertions/Test	assert/verify-like per test	proxy strutturale, non fault detection
Dimensione	TestLOC / LogicLOC / SupportLOC	volume e scaffolding	non confrontare raw LOC come qualità assoluta
Manutenibilità	Sonar smell + technical debt	rilevi e costo stimato sul test code	interpretare insieme a LOC/scaffolding
Categorie Sonar	Clean Code attributes	CONSISTENT / INTENTIONAL / ADAPTABLE / RESPONSIBLE	aggregati descrittivi, non classifica unica

Tabella 12 - Rilevanza funzionale e limiti osservati delle tecniche automatiche

Classe	Tecnica	Comportamento / oracle osservato	Evidenza quantitativa C0	Limite principale
UIHelpers	RND	sequenze API eseguibili con regressione su valori osservati	35/35; Line 10,25%; Branch 12,40%; MS 9,47%	reachability e discrimination limitate
UIHelpers	ES	ricerca search-based di percorsi strutturali	40/43; Line 23,02%; Branch 22,41%; PIT N/A	suite non fully passing
UIHelpers	LLM	oracle semantici concentrati su comportamenti leggibili dal contesto	35/35; Line 7,13%; TS 98%	porzione di codice raggiunta ridotta
RedisFilterBolt	RND	invocazioni API senza forte semantica emit/ack	11/11; Method 100%; Branch 4,76%; MS 0%	raggiunge metodi ma non discrimina i mutanti
RedisFilterBolt	ES	ricerca strutturale con alcuni oracle efficaci	9/9; Line 35,56%; TS 66,67%	pochi mutanti raggiunti
RedisFilterBolt	LLM	dispatch per data type + outcome emit/ack/fail	11/11; Line 84,44%; Branch 80,95%; MS/TS 75%	non garantisce copertura completa né stabilità cross-variant

Appendice C - Confronto C0-C4

La matrice finale distingue tre livelli: test forniti al refactoring, riuso delle suite **C0** per il behavior preservation e nuove suite automatiche generate sulle varianti congelate.

Tabella 13 - Versioni C0-C4 e test forniti al refactoring

Versione	Stato	Test BB	Test CF	Test MT	Test RND	Test ES	Test LLM
C0	Originale	N/A	N/A	N/A	N/A	N/A	N/A
C1	Refactored	No	No	No	No	No	No
C2	Refactored	Si	No	No	No	No	No
C3	Refactored	Si	Si	No	No	No	No
C4 UIHelpers	Refactored	Si	Si	Si	No	No	No
C4 RedisFilterBolt	Refactored	Si	Si	Si*	No	No	No

* Per **RedisFilterBolt** non esiste una suite **T_MT** aggiuntiva: **C4** riceve il set finale mutation-guided, coincidente con **T_BB+T_CF** (15 test). RND/ES/LLM vengono generati dopo il freeze e non sono input del refactoring.

Tabella 14 - UIHelpers: suite C0 eseguite su C0-C4 (behavior preservation)

Test eseguiti	C0	C1	C2	C3	C4
Test BB	P 35/35	P 35/35	P 35/35	P 35/35	P 35/35
Test CF	P 45/45	P 45/45	P 45/45	P 45/45	P 45/45
Test MT	P 53/53	F 52/53	F 52/53	P 53/53	F 52/53
Test LLM C0	P 35/35	P 35/35	P 35/35	P 35/35	P 35/35
Test RND C0	P 35/35	P 35/35	P 35/35	P 35/35	P 35/35
Test ES C0	F 40/43*	N/V 39/43*	N/V 38/43*	N/V 39/43*	N/V 39/43*

P=pass; F=fail; N/V=preservation non valutabile. La baseline **EvoSuite C0** non è fully passing (40/43), quindi i conteggi **C1-C4** restano diagnostici.

Tabella 15 - RedisFilterBolt: suite C0 eseguite su C0-C4 (behavior preservation)

Test eseguiti	C0	C1	C2	C3	C4
Test BB	P 11/11	P 11/11	P 11/11	P 11/11	P 11/11
Test CF	P 15/15	P 15/15	P 15/15	P 15/15	P 15/15
Test MT (=CF)	P 15/15	P 15/15	P 15/15	P 15/15	P 15/15
Test LLM C0	P 11/11	P 11/11	P 11/11	P 11/11	P 11/11
Test RND C0	P 11/11	P 11/11	P 11/11	P 11/11	P 11/11
Test ES C0	P 9/9	P 9/9	P 9/9	P 9/9	P 9/9

Tabella 16 - UIHelpers: delta delle nuove suite automatiche C1-C4 rispetto a C0

Var.	Tecnica	Delta Line pp	Delta Branch pp	Delta MS pp	Delta TS pp	Delta TestLOC	Delta Assert/Test	Delta Name pp	Delta Smell	Delta Debt
C1	RND	+0,05	0,00	+0,11	0,00	0	0,00	0	0	0
C2	RND	+1,31	-0,41	-1,08	-8,23	+20	+0,51	0	-1	+2
C3	RND	+0,06	0,00	+0,11	0,00	0	0,00	0	0	0
C4	RND	+0,06	0,00	+0,11	0,00	0	0,00	0	0	0
C1	ES	-5,39	-1,75	N/A	N/A	-299	+0,05	0	-14	-70
C2	ES	-2,59	-0,10	N/A	N/A	-213	+0,05	0	-17	-110
C3	ES	-0,35	+3,21	N/A	N/A	-197	+0,05	0	-9	+22
C4	ES	+2,25	+2,79	N/A	N/A	-179	+0,00	0	-10	+9
C1	LLM	N/A	N/A	N/A	N/A	-122	+0,20	0	0	0
C2	LLM	-0,05	-1,65	N/A	N/A	-145	-0,03	0	+36	+176
C3	LLM	+1,56	+0,41	N/A	N/A	-125	+0,29	0	+3	+3
C4	LLM	N/A	N/A	N/A	N/A	-145	+0,11	0	0	0

N/A indica misura dinamica non comparabile. Delta Name pp è la variazione della percentuale di nomi descrittivi.

Tabella 17 - RedisFilterBolt: delta delle nuove suite automatiche C1-C4 rispetto a C0

Var.	Tecnica	Delta Line pp	Delta Branch pp	Delta MS pp	Delta TS pp	Delta TestLOC	Delta Assert/Test	Delta Name pp	Delta Smell	Delta Debt
C1	RND	0,00	0,00	0,00	0,00	-21	-0,18	0	-20	-76
C2	RND	+10,09	0,00	0,00	0,00	-21	-0,18	0	-20	-76
C3	RND	+9,25	-0,41	0,00	0,00	-21	-0,18	0	-20	-76
C4	RND	+7,93	-0,41	0,00	0,00	-21	-0,18	0	-20	-76
C1	ES	-2,22	-4,76	-16,67	-66,67	-37	-0,01	0	-4	-37
C2	ES	+10,60	0,00	N/A	N/A	-169	0,00	0	-1	-11
C3	ES	+8,63	-6,00	-16,67	-66,67	+89	-0,01	0	-5	-39
C4	ES	+7,63	-6,00	-16,67	-66,67	-35	-0,11	0	-1	-11
C1	LLM	-4,44	+4,76	0,00	+6,82	0	+0,36	0	+10	+54
C2	LLM	-4,96	0,00	+3,26	+3,26	+13	+0,45	0	+12	+56
C3	LLM	-3,05	-7,04	-3,00	-3,00	+6	+0,36	0	+10	+54
C4	LLM	-2,63	+6,00	+17,00	+17,00	+6	+0,27	0	+10	+54

Tabella 18 - Unità PIT N/A e relativo blocker

Classe	Var.	Tecnica	Blocker
UIHelpers	C0	ES	SUITE_NOT_FULLY_PASSING
UIHelpers	C1	ES	SUITE_NOT_FULLY_PASSING
UIHelpers	C1	LLM	N/A_COMPILE_BLOCKER
UIHelpers	C2	ES	SUITE_NOT_FULLY_PASSING

Classe	Var.	Tecnica	Blocker
UIHelpers	C2	LLM	SUITE_NOT_FULLY_PASSING
UIHelpers	C3	ES	SUITE_NOT_FULLY_PASSING
UIHelpers	C3	LLM	SUITE_NOT_FULLY_PASSING
UIHelpers	C4	ES	SUITE_NOT_FULLY_PASSING
UIHelpers	C4	LLM	N/A_COMPILE_BLOCKER
RedisFilterBolt	C2	ES	SUITE_NOT_FULLY_PASSING

PIT è MEASURED in 20/30 unità. Le 10 unità sopra rimangono **N/A** e non vengono sostituite con zero.

Tabella 19 - Sonar del test code: smell, debt e categorie Clean Code sulle 30 unità

Tecnica	Code smell	Debt min	CONSISTENT	INTENTIONAL	ADAPTABLE	RESPONSIBLE
RND	1184	10888	21	928	235	0
ES	499	6103	108	376	15	0
LLM	96	1902	79	12	5	0
Totale	1779	18893	208	1316	255	0

I conteggi sono aggregati sul codice di test generato, non sul production code. Le differenze di NCLOC/scaffolding impediscono di interpretarli come una classifica normalizzata.

Tabella 20 - Matrice automatica UIHelpers C0-C4

Var.	Tecnica	Pass/Exec	Line %	Branch %	MS %	TS %	Smell	Debt
C0	RND	35/35	10,2496	12,3967	9,4675	45,0704	183	1432
C0	ES	40/43	23,0216	22,4138	N/A	N/A	84	1023
C0	LLM	35/35	7,1301	15,2893	14,497	98	0	0
C1	RND	35/35	10,2954	12,3967	9,5808	45,0704	183	1432
C1	ES	41/43	17,6365	20,6612	N/A	N/A	70	953
C1	LLM	0/0	N/A	N/A	N/A	N/A	0	0
C2	RND	35/35	11,5591	11,9835	8,3832	36,8421	182	1434
C2	ES	35/41	20,4301	22,3140	N/A	N/A	67	913
C2	LLM	34/35	7,0789	13,6364	N/A	N/A	36	176
C3	RND	35/35	10,3047	12,3967	9,5808	45,0704	183	1432
C3	ES	40/48	22,6703	25,6198	N/A	N/A	75	1045
C3	LLM	32/35	8,6918	15,7025	N/A	N/A	3	3
C4	RND	35/35	10,3047	12,3967	9,5808	45,0704	183	1432
C4	ES	38/45	25,2688	25,2066	N/A	N/A	74	1032
C4	LLM	0/0	N/A	N/A	N/A	N/A	0	0

Tabella 21 - Matrice automatica RedisFilterBolt C0-C4

Var.	Tecnica	Pass/Exec	Line %	Branch %	MS %	TS %	Smell	Debt
C0	RND	11/11	48,8889	4,7619	0	0	70	806
C0	ES	9/9	35,5556	19,0476	16,6667	66,6667	28	247
C0	LLM	11/11	84,4444	80,9524	75	75	3	301
C1	RND	11/11	48,8889	4,7619	0	0	50	730
C1	ES	8/8	33,3333	14,2857	0	0	24	210

Var.	Tecnica	Pass/Exec	Line %	Branch %	MS %	TS %	Smell	Debt
C1	LLM	11/11	80	85,7143	75	81,8182	13	355
C2	RND	11/11	58,9744	4,7619	0	0	50	730
C2	ES	6/9	46,1538	19,0476	N/A	N/A	27	236
C2	LLM	11/11	79,4872	80,9524	78,2609	78,2609	15	357
C3	RND	11/11	58,1395	4,3478	0	0	50	730
C3	ES	8/8	44,1860	13,0435	0	0	23	208
C3	LLM	11/11	81,3953	73,9130	72	72	13	355
C4	RND	11/11	56,8182	4,3478	0	0	50	730
C4	ES	9/9	43,1818	13,0435	0	0	27	236
C4	LLM	11/11	81,8182	86,9565	92	92	13	355

Appendice D - Tracciabilità

Tabella 22 - Difficoltà tecniche, soluzione e principio preservato

Difficoltà	Rischio sperimentale	Soluzione adottata	Principio preservato
EvoSuite 1.2.0 vs Java 25 / Multi-Release JAR	confondere incompatibilità framework e difetto della suite	percorso Java 11 di compatibilità e classpath preprocessato senza cambiare il SUT	framework failure != test failure
Parser PIT e decimali con virgola	ricostruire summary errati da evidence valide	correzione del parser sui raw artefacts senza rieseguire PIT	raw evidence come source of truth
LLM Redis con external grounding	contaminazione del contesto preregistrato	tentativo 1 invalidato; nuova conversazione controllata	nessuna fonte esterna non autorizzata
Suite non fully passing / compile blocker	trasformare N/A in zero o riparare post-freeze	stato tecnico preservato e PIT bloccato quando non valido	N/A != 0; no tuning opportunistico
Regression CORS MT06	nascondere un esito negativo del refactoring	fallimento mantenuto su C1/C2/C4 e confrontato con C3	behavior preservation come gate indipendente
Coverage finale UIHelpers T_MT	riaprire la progettazione dopo la misura	JaCoCo post-hoc sul set congelato 53/53, hash/HEAD invariati	misura descrittiva, nessun feedback retroattivo

Tabella 23 - Artefatti canonici usati per la tracciabilità

Area	Artefatto	Uso
Quadro testing	isw2/testing/README.md	organizzazione, classi e criteri di lettura
Class selection	isw2/docs/testing-class-selection.md	ranking, filtro e target
Second class reassessment	isw2/docs/testing-second-class-reassessment.md	archivio DefaultHttpCredentialsPlugin e scelta RedisFilterBolt
UI Category Partition	isw2/testing/specs/uihelpers/category-partition.md	famiglie, scelte, vincoli e boundary
UI oracle audit	isw2/testing/specs/uihelpers/oracle-source-audit.csv	provenienza degli oracle T_BB
Redis Category Partition	isw2/testing/specs/redis-filter-bolt/category-partition.md	categorie C1-C5 e frame
UI results	isw2/testing/results/uihelpers/README.md	BB, CF, MT, RND e reliability
Redis final testing	isw2/testing/results/redis-filter-bolt/final-report.md	BB, CF, RND, ES, LLM e PIT
Reliability	isw2/testing/results/*/reliability/	profili uniformi, summary ed estimator
Behavior preservation	isw2/testing/results/m4-generated/behavior-preservation/	suite C0 su C1-C4
30-unit matrix	isw2/testing/results/m4-generated/final-deangelis-consolidation/deangelis-30-unit-integrated-matrix.csv	coverage, PIT, struttura e Sonar
Delta C1-C4	isw2/testing/results/m4-generated/final-deangelis-consolidation/deangelis-c1-c4-deltas-vs-c0.csv	delta per tecnica rispetto a C0
Sonar final campaign	isw2/testing/results/m4-generated/sonar/final-campaign-v5-resume/sonar-30-unit-final-matrix.csv	30/30 unità di test code
Classi target	isw2/testing/classes.txt	FQCN finali
CI	/.github/workflows/isw2-testing.yml	build, integrity e final gate

Classi target congelate

org.apache.storm.daemon.ui.**UIHelpers**

org.apache.storm.redis.bolt.**RedisFilterBolt**

*I valori **N/A**, i compile blocker, i framework failure, il no-op Redis **C1** e il fallimento **MT06** sono mantenuti come risultati sperimentali distinti. Non vengono reinterpretati come zero e non vengono rimossi dalle evidence.*